University of Calgary
CPSC 433 – Artificial Intelligence

Course Timetabling via Two Alternative Search Pipelines

OR-Tree Feasibility followed by
Set-Based or AND-Tree Optimization

Formal Search Models and Processes – Proposal

Member 1: Syed Essam Uddin Khawaja 30242943

Member 2: [NAME / UCID]

Member 3: [NAME / UCID]

Member 4: [NAME / UCID]

Member 5: [NAME / UCID]

Date: [SUBMISSION DATE]

Abstract

We consider the CPSC 433 course timetabling problem: assigning lectures and tutorials to weekly slots while satisfying hard constraints and minimizing penalties for soft constraints. We propose two alternative search systems that share an OR-tree feasibility stage. The OR-tree represents partial assignments as problem descriptions and applies depth-first search with minimum remaining values to find an initial valid timetable. In Proposal A, set-based search represents the timetable as assignment facts and applies valid Move and Swap extensions using greedy hill climbing. In Proposal B, AND-tree search divides the optimization problem into assignment cases and uses branch-and-bound with the feasible timetable as an initial incumbent. Both alternatives preserve hard constraints and can improve the initial evaluation score. The set-based process is simpler but can stop at a local optimum; the AND-tree process can establish global optimality if exhaustive exploration and sound pruning complete. We specify the models, controls, stopping conditions, and worked examples using the terminology of the project problem description and course lectures.

1 Introduction

The objective is to assign lectures and tutorials to permitted weekly slots so that every hard constraint holds and the soft-constraint penalty is minimized. This proposal compares two ways to improve a valid timetable: OR-tree feasibility followed by set-based local search, or OR-tree feasibility followed by AND-tree branch-and-bound. Section 2 formalizes the scheduling instance once; later sections instantiate the search models and controls without redefining the same constraints.

2 Formal Scheduling Problem

We use the terminology and symbols of the problem description [2] throughout. Symbols it does not define, such as *overlap*, $D(x)$ and Consistent, are introduced below.

2.1 Components, slots, and domains

Components: *Lectures* is the set of lecture sections and *Tutorials* the set of tutorials and labs; labs (`LAB`) are treated exactly like tutorials (`TUT`). The set of all components to schedule is $X = \mathit{Lectures} + \mathit{Tutorials}$. Each lecture is named *program, course number, section*, for example `CPSC 433 LEC 01`. For $x \in X$, $\mathit{course}(x)$ is the pair (program, number), e.g. (`CPSC`, 433).

Which lectures a tutorial belongs to: A tutorial must not clash with its lecture. The tutorial's name tells us which lecture that is, and there are two naming forms:

- `CPSC 433 LEC 01 TUT 02` names a section. It belongs to `CPSC 433 LEC 01` only.
- `CPSC 433 TUT 01` names no section. It is a shared tutorial, open to students of every section, so it belongs to all lectures of `CPSC 433`.

The set of lectures that tutorial t belongs to is

$$\mathit{Lec}(t) = \begin{cases} \{\text{the lecture named in } t\} & \text{if } t\text{'s name has a } \texttt{LEC} \text{ part,} \\ \{\, l \in \mathit{Lectures} : \mathit{course}(l) = \mathit{course}(t) \,\} & \text{otherwise (shared tutorial).} \end{cases} \tag{1}$$

Weekly slots: The UofC rules link days together. A lecture placed on Monday must also meet Wednesday and Friday at the same time; anything placed on Tuesday must also meet Thursday; a

tutorial placed on Monday must also meet Wednesday. So the scheduler never picks single days. It picks a *day pattern and a start time*, and the linked days follow automatically. We call one such choice a *slot*. Because the linking is built into the slot, the day-linking rules always hold and do not appear among the hard constraints in Section 2.2.

Formally, a slot is a triple

$$s = (\mathit{days}(s),\ \mathit{start}(s),\ \mathit{end}(s)), \tag{2}$$

meaning "meet on every day in $\mathit{days}(s)$, from $\mathit{start}(s)$ to $\mathit{end}(s)$." For example, the lecture slot used for a Monday 9:00 lecture is $(\{\text{Mon}, \text{Wed}, \text{Fri}\},\ 9{:}00,\ 10{:}00)$. For short, we name a slot by its day pattern and start time, so this slot is written *MWF 9:00*. The day patterns are written MWF (Mon/Wed/Fri), TT (Tue/Thu), MW (Mon/Wed) and F (Fri only).

Lectures and tutorials use different day patterns; Table 1 lists every slot pattern.

Table 1: Slot patterns. Each row is one group of slots; h is the start time.

Used by	Written as	Meets on	Length	Start times	Count
Lectures	MWF h	Mon, Wed, Fri	1 h	8:00, 9:00, …, 20:00	13
Lectures	TT h	Tue, Thu	1.5 h	8:00, 9:30, …, 18:30	8
Tutorials	MW h	Mon, Wed	1 h	8:00, 9:00, …, 20:00	13
Tutorials	TT h	Tue, Thu	1 h	8:00, 9:00, …, 20:00	13
Tutorials	F h	Fri only	2 h	8:00, 10:00, …, 18:00	6

A given input offers only some of these slots, each with its capacities. *Slots* is the set of offered slots. The offered lecture slots (top half of Table 1) form Slots_L and the offered tutorial slots (bottom half) form Slots_T, so $\mathit{Slots} = \mathit{Slots}_L + \mathit{Slots}_T$. Lectures may only use Slots_L and tutorials only Slots_T.

Time overlap: Since lecture and tutorial slots follow different grids, we compare actual meeting times. Two slots s and s' *overlap* when they meet on a common day and their time ranges intersect:

$$\mathit{overlap}(s, s') \iff \mathit{days}(s) \cap \mathit{days}(s') \neq \varnothing \ \wedge\ \mathit{start}(s) < \mathit{end}(s') \ \wedge\ \mathit{start}(s') < \mathit{end}(s). \tag{3}$$

For example, lecture slot TT 9:30 (9:30-11:00) overlaps tutorial slots TT 9:00 and TT 10:00.

Two lectures, or two tutorials, can only clash if they are in the same slot, because slots of the same kind either fall on different days or follow each other without overlapping:

$$s, s' \in \mathit{Slots}_L \ \text{ or } \ s, s' \in \mathit{Slots}_T \implies (\mathit{overlap}(s, s') \iff s = s'). \tag{4}$$

Domains: The *domain* $D(x)$ is the set of slots component x may use: the slots of its kind (Slots_L or Slots_T), minus any slot ruled out by a hard constraint on x alone (C_5, C_8 to C_{10} and C_{12} in Section 2.2).

Assignments: A complete timetable is a function

$$\mathit{assign} : \mathit{Lectures} + \mathit{Tutorials} \to \mathit{Slots}, \qquad \mathit{assign}(x) \in D(x) \text{ for every } x \in X. \tag{5}$$

While a timetable is being built, only some components have a slot yet. Such a *partial assignment* *assign* is defined only on the components $\mathrm{dom}(\mathit{assign}) \subseteq X$ placed so far. (This is different from the input preassignment *partassign* in Section 2.2.)

Slot counts: The capacity constraints (C_1 to C_4) need to know how full each slot is. For an assignment *assign* and a slot s, $n_L(s)$ is the number of lectures *assign* puts in s and $n_T(s)$ the number of tutorials.

$n_{L,AL}(s)$ and $n_{T,AL}(s)$ count only the ones with an *activelearning* statement. Only placed components are counted, so the same counts work for partial assignments:

$$\begin{aligned} n_L(s) &= |\{l \in \mathit{Lectures} \cap \mathrm{dom}(\mathit{assign}) : \mathit{assign}(l) = s\}|, \\ n_T(s) &= |\{t \in \mathit{Tutorials} \cap \mathrm{dom}(\mathit{assign}) : \mathit{assign}(t) = s\}|, \\ n_{L,AL}(s) &= |\{l \in \mathit{Lectures} \cap \mathrm{dom}(\mathit{assign}) : \mathit{activelearning}(l),\ \mathit{assign}(l) = s\}|, \\ n_{T,AL}(s) &= |\{t \in \mathit{Tutorials} \cap \mathrm{dom}(\mathit{assign}) : \mathit{activelearning}(t),\ \mathit{assign}(t) = s\}|. \end{aligned}$$

For example, if *assign* puts `CPSC 433 LEC 01` and `DATA 201 LEC 01` in MWF 9:00 and only `DATA 201 LEC 01` needs active learning, then $n_L(\text{MWF 9:00}) = 2$ and $n_{L,AL}(\text{MWF 9:00}) = 1$. C_1 then checks $2 \le \mathit{lecturemax}(\text{MWF 9:00})$ and C_3 checks $1 \le \mathit{ALlecturemax}(\text{MWF 9:00})$.

2.2 Hard constraints

Hard constraints are the rules a valid timetable cannot break. Besides the sets defined above, they use these inputs:

- lecture capacities $\mathit{lecturemax}(s)$ and $\mathit{ALlecturemax}(s)$ for each $s \in \mathit{Slots}_L$;
- tutorial capacities $\mathit{tutorialmax}(s)$ and $\mathit{ALtutorialmax}(s)$ for each $s \in \mathit{Slots}_T$;
- the $\mathit{notcompatible}(x, y)$ and $\mathit{unwanted}(x, s)$ statements;
- the preassignment $\mathit{partassign} : \mathit{Lectures} + \mathit{Tutorials} \to \mathit{Slots} + \{\$\}$, where \$ means no preassignment;
- the $\mathit{activelearning}(x)$ statements.

Here $\mathit{ALmax}(s)$ means $\mathit{ALlecturemax}(s)$ if $s \in \mathit{Slots}_L$ and $\mathit{ALtutorialmax}(s)$ if $s \in \mathit{Slots}_T$. Each constraint only looks at components that have been placed, so it can also be checked on a partial assignment.

$$\begin{aligned}
C_1 &: \forall s \in \mathit{Slots}_L : \ n_L(s) \le \mathit{lecturemax}(s) && \text{(lecture capacity)} && (6)\\
C_2 &: \forall s \in \mathit{Slots}_T : \ n_T(s) \le \mathit{tutorialmax}(s) && \text{(tutorial capacity)} && (7)\\
C_3 &: \forall s \in \mathit{Slots}_L : \ n_{L,AL}(s) \le \mathit{ALlecturemax}(s) && \text{(AL lecture capacity)} && (8)\\
C_4 &: \forall s \in \mathit{Slots}_T : \ n_{T,AL}(s) \le \mathit{ALtutorialmax}(s) && \text{(AL tutorial capacity)} && (9)\\
C_5 &: \forall x \text{ with } \mathit{activelearning}(x) : \ \mathit{ALmax}(\mathit{assign}(x)) > 0 && \text{(AL placement)} && (10)\\
C_6 &: \forall t \in \mathit{Tutorials},\ \forall l \in \mathit{Lec}(t) : \ \neg\, \mathit{overlap}(\mathit{assign}(l), \mathit{assign}(t)) && \text{(lecture/tutorial)} && (11)\\
C_7 &: \forall\, \mathit{notcompatible}(x, y) : \ \neg\, \mathit{overlap}(\mathit{assign}(x), \mathit{assign}(y)) && \text{(not compatible)} && (12)\\
C_8 &: \forall x \text{ with } \mathit{partassign}(x) \ne \$: \ \mathit{assign}(x) = \mathit{partassign}(x) && \text{(preassignment)} && (13)\\
C_9 &: \forall\, \mathit{unwanted}(x, s) : \ \mathit{assign}(x) \ne s && \text{(unwanted)} && (14)\\
C_{10} &: \forall x \in X_9 : \ \mathit{start}(\mathit{assign}(x)) \ge 18{:}00 && \text{(LEC 9 evening)} && (15)\\
C_{11} &: \forall l \ne l' \in L_5 : \ \mathit{assign}(l) \ne \mathit{assign}(l') && \text{(5XX non-overlap)} && (16)\\
C_{12} &: \forall l \in \mathit{Lectures} : \ \mathit{assign}(l) \ne \text{TT 11:00} && \text{(TT 11:00 meeting)} && (17)\\
C_{13} &: \text{special CPSC 851/913 bookings (below)}
\end{aligned}$$

X_9 is the set of `LEC 9` sections and their tutorials (e.g. `CPSC 433 LEC 90 TUT 01`), and L_5 is the set of 5XX lectures across all programs. The problem description writes C_6 as $\mathit{assign}(l) \ne \mathit{assign}(t)$; since lecture and tutorial slots are never equal, we read it as "must not overlap".

Special bookings (C_{13}): CPSC 851 and CPSC 913 never appear in the input; we create them. If the input has a CPSC 351 lecture, we add the tutorial t_{851} = `CPSC 851 TUT 01` to *Tutorials*, set $\mathit{partassign}(t_{851})$ = TT 18:00, and add $\mathit{notcompatible}(t_{851}, y)$ for every CPSC 351 lecture and tutorial y. CPSC 913 is created the same way, as t_{913}, when the input has a CPSC 413 lecture. C_{13} is then checked by C_7 and C_8. Like any other tutorial, these bookings count toward $\mathit{tutorialmax}$(TT 18:00), so if the input does not offer TT 18:00 with enough room for them, no valid timetable exists.

Validity: A complete timetable *assign* is valid if it breaks no hard constraint:

$$\boxed{\text{Constr}(\mathit{assign}) = \bigwedge_{i=1}^{13} C_i(\mathit{assign}).} \tag{18}$$

A partial assignment is *consistent* if the components placed so far break no constraint:

$$\text{Consistent}(\mathit{assign}) \iff \text{every } C_i \text{ holds on } \text{dom}(\mathit{assign}). \tag{19}$$

Once a partial assignment breaks a rule, placing more components cannot fix it: a full slot stays full and a clash stays a clash. So every model can discard a partial assignment as soon as Consistent fails.

2.3 Soft constraints and objective

Soft constraints may be broken, but each violation adds penalty points. They use these inputs:

- minimums $\mathit{lecturemin}(s)$ for $s \in \mathit{Slots}_L$ and $\mathit{tutorialmin}(s)$ for $s \in \mathit{Slots}_T$;
- $\mathit{preference} : (\mathit{Lectures} + \mathit{Tutorials}) \times \mathit{Slots} \to \mathbb{N}$ (0 where none is given);
- the $\mathit{pair}(x, y)$ statements, where x and y are either two lectures or two tutorials;
- penalties $\mathit{pen}_{\mathit{lecturemin}}$, $\mathit{pen}_{\mathit{tutorialmin}}$, $\mathit{pen}_{\mathit{notpaired}}$, $\mathit{pen}_{\mathit{section}} \in \mathbb{N}$.

There are five penalties:

Too few lectures: If a lecture slot holds fewer lectures than its minimum, each missing lecture costs $\mathit{pen}_{\mathit{lecturemin}}$. A slot with minimum 3 that holds 1 lecture costs $2 \cdot \mathit{pen}_{\mathit{lecturemin}}$; a slot at or above its minimum costs nothing.

$$E_{L\,\min}(\mathit{assign}) = \mathit{pen}_{\mathit{lecturemin}} \cdot \sum_{s \in \mathit{Slots}_L} \max(0,\ \mathit{lecturemin}(s) - n_L(s)) \tag{20}$$

Too few tutorials: The same for tutorial slots.

$$E_{T\,\min}(\mathit{assign}) = \mathit{pen}_{\mathit{tutorialmin}} \cdot \sum_{s \in \mathit{Slots}_T} \max(0,\ \mathit{tutorialmin}(s) - n_T(s)) \tag{21}$$

Unmet preferences: If a component does not get a slot it has a preference for, that preference's value is added.

$$E_{\text{pref}}(\mathit{assign}) = \sum_{x \in X} \sum_{s \in \mathit{Slots},\ s \neq \mathit{assign}(x)} \mathit{preference}(x, s) \tag{22}$$

Unmet pairs: If the two components of a $\mathit{pair}(x, y)$ statement are not in the same slot, this costs $\mathit{pen}_{\mathit{notpaired}}$. The order does not matter: $\mathit{pair}(x, y)$ and $\mathit{pair}(y, x)$ count as one preference.

$$E_{\text{pair}}(\mathit{assign}) = \mathit{pen}_{\mathit{notpaired}} \cdot \left|\{\, \{x, y\} : \mathit{pair}(x, y),\ \mathit{assign}(x) \neq \mathit{assign}(y) \,\}\right| \tag{23}$$

Sections sharing a slot: Each pair of lecture sections of the same course placed in the same slot costs $pen_{section}$.

$$E_{\text{section}}(assign) = pen_{section} \cdot |\{\, \{l, l'\} \subseteq Lectures : l \neq l',\ course(l) = course(l'),\ assign(l) = assign(l')\,\}| \tag{24}$$

Objective: Eval is the total penalty,

$$\boxed{\begin{aligned}\text{Eval}(assign) = E_{L\,\min}(assign) + E_{T\,\min}(assign) + E_{\text{pref}}(assign) \\ + E_{\text{pair}}(assign) + E_{\text{section}}(assign),\end{aligned}} \tag{25}$$

and the problem is

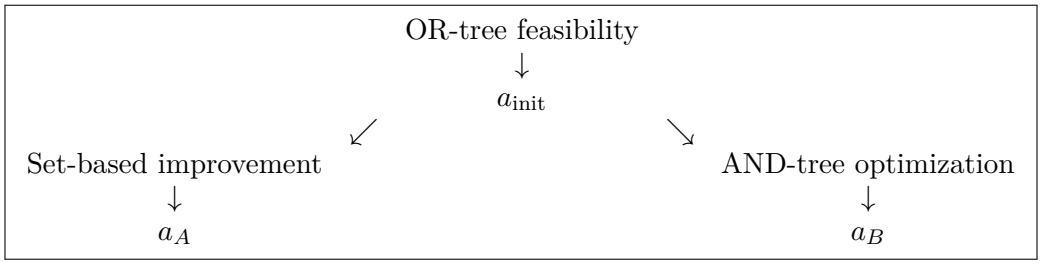
$$\boxed{\min_{assign} \text{Eval}(assign) \quad \text{subject to} \quad \text{Constr}(assign) = \mathsf{true}.} \tag{26}$$

Every penalty is 0 or more, so a valid timetable with $\text{Eval}(assign) = 0$ is optimal.

3 Overall Search Architecture

We propose two complete search systems that share the same first stage:

- **Proposal A:** OR-tree for an initial valid timetable (Section 4) → set-based improvement (Section 5).
- **Proposal B:** OR-tree for an initial valid timetable (Section 4) → AND-tree optimization (Section 6).



All three models use the problem definition of Section 2: the components X, the domains $D(x)$, the overlap test (3), the checker Consistent (19) and Eval.

Shared first stage: Both second stages need a valid timetable to start from. The OR-tree finds one, a_{init}, or exhausts its alternatives. It only looks for feasibility and ignores Eval.

How the stages connect: Both second stages begin with a_{init} as their best timetable and only replace it with a strictly better valid one, so

$$\text{Eval}(a_A) \leq \text{Eval}(a_{\text{init}}), \qquad \text{Eval}(a_B) \leq \text{Eval}(a_{\text{init}}).$$

4 Model I: OR-Tree Feasibility Search

4.1 Search model: $A_\vee = (S_\vee, T_\vee)$

Problem descriptions.

$$\boxed{Prob_\vee = \{(a, U) : a \text{ is a consistent partial assignment},\ U = X \setminus \text{dom}(a)\}.}$$

Here a is the current partial timetable, and U contains unscheduled components. A node has the lecture's form $((a, U), sol)$, where $sol \in \{\mathsf{yes}, ?, \mathsf{no}\}$.

Alternatives. Select a component $x \in U$ and define

$$D_a(x) = \{s \in D(x) : \text{Consistent}(a \cup \{x \mapsto s\})\}.$$

For every $s_i \in D_a(x)$, let $pr_i = (a \cup \{x \mapsto s_i\}, U \setminus \{x\})$. Then the application-specific alternatives relation is

$$\boxed{\mathit{Altern}((a,U), pr_1, \ldots, pr_r), \qquad r = |D_a(x)|.}$$

For every selected x, all its currently legal slot alternatives are represented; the control tries them until one succeeds or all fail.

States and extension rules. Following Lecture 11, $S_\vee \subseteq \mathit{Otree}$ and $T_\vee \subseteq S_\vee \times S_\vee$. We instantiate $\mathit{Erw}_\vee$ by the following leaf operations, which may also be performed within a larger tree:

$$\begin{aligned}
((a,\varnothing),?) &\longrightarrow ((a,\varnothing),\mathsf{yes}) && \text{if Constr}(a),\\
((a,U),?) &\longrightarrow ((a,U),\mathsf{no}) && \text{if no valid completion is possible,}\\
(pr,?) &\longrightarrow (pr,?,(pr_1,?),\ldots,(pr_r,?)) && \text{if } \mathit{Altern}(pr, pr_1, \ldots, pr_r).
\end{aligned}$$

The second case applies when the partial assignment violates a constraint or some remaining component has $D_a(x) = \varnothing$; an internal node is marked unsuccessful only after all child alternatives have failed. $\mathit{Erw}^*_\vee$ allows an applicable extension inside an OR-tree while the rest of the tree is unchanged. Thus transitions apply either relation, and only one successful branch is required.

4.2 Search process: $P_\vee = (A_\vee, \mathit{Env}_\vee, K_\vee)$

The environment stores the scheduling instance, the current search tree, and a deterministic order for components and slots. The control selects the deepest unresolved leaf (depth-first search), then uses minimum remaining values (MRV):

$$\boxed{x^* = \arg\min_{x \in U} |D_a(x)|.}$$

Ties are broken by the fixed component order; feasible slots are tried in a fixed slot order. If $D_a(x^*) = \varnothing$, the leaf fails. Otherwise the control expands it with all currently legal alternatives. No forward checking is necessary for correctness.

4.3 Search instance: $\mathit{Inst}_\vee = (q_0, G_\vee)$

The initial state is $q_0 = ((\varnothing, X), ?)$. For a tree q, define success recursively:

$$\begin{aligned}
\text{Success}((pr,\mathsf{yes})) &= \mathsf{true},\\
\text{Success}((pr,\mathsf{no})) &= \mathsf{false},\\
\text{Success}((pr,?,b_1,\ldots,b_r)) &= \bigvee_{i=1}^{r} \text{Success}(b_i).
\end{aligned}$$

$G_\vee(q) = \mathsf{yes}$ when a successful branch exists, or when every branch has been resolved unsuccessful. These are distinct outcomes: the former returns a complete valid timetable; the latter returns FAILURE. A resource limit reached before either outcome is inconclusive.

4.4 Implementation and operation

The search uses a **dictionary** a of assigned components and a **set** U of unscheduled components. Domains are stored as lists of slot identifiers; occupancy counts and overlapping slots are cached in arrays or dictionaries for fast constraint checks. A **depth-first stack** represents the active OR-tree path. Each frame stores (a, U) (or a reversible assignment trail), the MRV-selected component, untried legal slots and its status $\{?, \mathsf{yes}, \mathsf{no}\}$. The control tries a slot, updates the counts and checks consistency; after failure it undoes the assignment and tries the next alternative. It returns the first complete valid timetable, or FAILURE only after all alternatives are exhausted.

4.5 Worked OR-tree derivation

Use the three-lecture instance in Section 6.3. Let the fixed offering order be $x < y < z$ and the OR-tree slot order be s_2, s_1. At the root, all three domains have size two; MRV chooses x. The first branch is

$$\begin{aligned}(\varnothing, \{x, y, z\}) &\to (\{x \mapsto s_2\}, \{y, z\}) \\ &\to (\{x \mapsto s_2, y \mapsto s_1\}, \{z\}) \\ &\to (\{x \mapsto s_2, y \mapsto s_1, z \mapsto s_2\}, \varnothing).\end{aligned}$$

The second step has only s_1 legal for y because x and y are not compatible. The final assignment satisfies capacities and all remaining hard constraints, so its leaf becomes yes and $G_\vee$ holds. The root's $x \mapsto s_1$ alternative remains unexplored: the OR-tree needs just one valid timetable, even though the feasible result has Eval = 5.

5 Model II: Set-Based Local Optimization

Now, we need a second model that will use the valid timetable produced by the OR-tree, and produce an output minimizing the soft penalties applied due to the soft constraints we have in the problem.

Set-based search works in this case because it allows for small improvements on the solution pass by pass.

5.1 Search model: $A_{set} = (S_{set}, T_{set})$

Our search model is $A_{set} = (S_{set}, T_{set})$. We first establish the facts that make up our timetables and the rules that allow us to modify them.

Facts. Following Lecture 06, we define our set of facts as:

$$F = \{\text{Assigned}(x, s) \mid x \in X,\ s \in D(x)\} \tag{27}$$

Each fact is simply one assignment in our timetable. By combining these facts, we can represent an entire timetable as a single set.

States. Lecture 06 defines the state space as a subset of the power set of facts:

$$S_{set} \subseteq 2^F \tag{28}$$

For our problem, we only want to consider states that represent complete, valid timetables. Therefore, we define:

$$S_{set} = \left\{ q \subseteq F \;\middle|\; \begin{array}{l} q \text{ assigns every } x \in X \text{ exactly once,} \\ \text{Constr}(a_q) = \text{true} \end{array} \right\} \tag{29}$$

Here, q represents a set of assignment facts, while a_q is the actual timetable represented by those facts. Every state in our search is already a valid solution; at this stage we improve assignments rather than schedule more components.

Extensions. Following Lecture 06, extensions replace facts: $EXT \subseteq \{A \longrightarrow B \mid A, B \subseteq F\}$. We propose two types of extensions: Move and Swap.

1. Move

A Move extension changes the time slot assigned to a single component.

$$\{\text{Assigned}(x, s_1)\} \longrightarrow \{\text{Assigned}(x, s_2)\} \tag{30}$$

Here, x is the component being moved, s_1 is its current time slot, and s_2 is the new time slot.

For example, we could move CPSC 231 LEC 02 from MWF 9–10 to MWF 10–11, provided that doing so does not violate any hard constraints.

2. Swap

A Swap extension exchanges the assigned slots of two components.

$$\begin{array}{l} \{\text{Assigned}(x, s_1), \text{Assigned}(y, s_2)\} \\ \quad \longrightarrow \{\text{Assigned}(x, s_2), \text{Assigned}(y, s_1)\} \end{array} \tag{31}$$

Here, x and y are two distinct components, while s_1 and s_2 are their current slots.

We use swaps because sometimes moving one component on its own may not be possible, but exchanging the slots of two components could still produce a better timetable.

For both extensions, we need to ensure that the new assignments belong to the components' allowed domains, respect preassignments, and do not violate any hard constraints.

This is important because we do not want our optimization process to undo the work already done by the OR-tree.

Transitions. Once we select an extension rule, we can apply it to the current state using the transition defined in Lecture 06:

$$q' = (q \setminus A) \cup B \tag{32}$$

Our transition relation is therefore:

$$\begin{array}{rl} T_{set} = \{(q, q') \mid & \exists (A \to B) \in EXT, \\ & A \subseteq q, \\ & q' = (q \setminus A) \cup B, \\ & q' \in S_{set}\} \end{array} \tag{33}$$

The last condition, $q' \in S_{set}$, ensures that every transition leads to another valid timetable.

5.2 Search process: $P_{set} = (A_{set}, Env_{set}, K_{set})$

Following Lecture 06, our search process is $P_{set} = (A_{set}, Env_{set}, K_{set})$.

Environment. Our environment contains the allowed slot domains, hard constraints, preferences and penalty values; its current configuration records q along with that fixed instance data.

Search control. For our control, we propose a greedy hill-climbing approach.

The idea is fairly simple. At every step, we consider the available Move and Swap extensions, calculate the resulting penalties, and choose the extension that gives us the lowest evaluation score.

However, we only apply an extension if it actually improves the current timetable.

Weight function

Lecture 06 uses f_{weight} to assign a numerical value to each applicable extension rule.

For our problem, we define:

$$f_{weight}(A, B, e) = \text{Eval}\left(a_{(q(e)\setminus A)\cup B}\right), \tag{34}$$

where $q(e)$ is the current state recorded in environment e.

The function essentially asks: if we perform this Move or Swap, what will the new timetable's penalty be?

Since our objective is to minimize penalties, a lower weight is preferable.

Applicable extensions

We define the set of currently applicable extensions as:

$$\begin{aligned} R(q) = \{(A, B) \mid\ & A \to B \in EXT, \\ & A \subseteq q, \\ & (q \setminus A) \cup B \in S_{set}, \\ & (q \setminus A) \cup B \neq q\} \end{aligned} \tag{35}$$

For nonempty $R(q)$, let

$$w_{\min}(q) = \min_{(A,B)\in R(q)} f_{weight}(A, B, e).$$

Selection function

It is possible that multiple extensions produce the same minimum weight.

In that case, we use the function f_{select} from Lecture 06 to break the tie.

First, we collect the best extensions:

$$R_{\min}(q) = \{(A, B) \in R(q) \mid f_{weight}(A, B, e) = w_{\min}(q)\} \tag{36}$$

We then select one:

$$(A^*, B^*) = f_{select}(R_{\min}(q), e) \tag{37}$$

Our proposed tie-breaker simply chooses randomly among the equally weighted extensions.

This gives us one extension to consider applying.

Strict improvement

We only apply an extension when $w_{\min}(q) < \text{Eval}(a_q)$. Then

$$K_{set}(q, e) = (q \setminus A^*) \cup B^*.$$

Otherwise, we stop the search and return the current timetable.

Local optimum. A local optimum is a valid timetable where no single Move or Swap can reduce the penalty. This does not necessarily mean we have found the best timetable possible.

5.3 Search instance: $Inst_{set} = (q_0, G_{set})$

The search instance is $Inst_{set} = (q_0, G_{set})$.

Initial state. Since the OR-tree has already produced a valid timetable, we do not need to start with an empty state.

Let a_{feasible} represent the timetable returned by the OR-tree.

We convert it into a set of assignment facts:

$$q_0 = \{\text{Assigned}(x, a_{\text{feasible}}(x)) \mid x \in X\} \tag{38}$$

Since $\text{Constr}(a_{\text{feasible}}) = \text{true}$, we know $q_0 \in S_{set}$.

Goal condition. Our goal is to stop at zero penalty, or when no improving neighbor exists. Let

$$N(q) = \{q' \in S_{set} : (q, q') \in T_{set},\ q' \neq q\}.$$

Then

$$G_{set}(q) = \textsf{yes} \iff \text{Eval}(a_q) = 0 \ \vee \ \neg\exists q' \in N(q) : \text{Eval}(a_{q'}) < \text{Eval}(a_q).$$

For larger instances, we could also introduce a time or iteration limit.

5.4 Implementation and operation

We start with the OR-tree's valid timetable, generate all possible valid neighbors through Move and Swap extensions, and select the best improvement. An implementation stores q as a **set of assignment facts** $\text{Assigned}(x, s)$ (equivalently, a dictionary from component to slot), $D(x)$ as a **dictionary of permitted slot lists**, and current room occupancies in **arrays or dictionaries**. Candidate Moves and Swaps can be enumerated using the component and slot lists, checked with Consistent and scored using Eval. Only the current state, the best candidate and its score are required; the search need not retain a tree of past states. If no candidate reduces the penalty, the current timetable is returned.

5.5 Worked set-based derivation

To demonstrate how our search works, consider a simplified scheduling problem involving two lecture sections and two tutorials for CPSC 231.

$$L_1 = \text{CPSC 231 LEC 01}, \quad L_2 = \text{CPSC 231 LEC 02},$$
$$T_1 = \text{CPSC 231 TUT 01}, \quad T_2 = \text{CPSC 231 TUT 02}.$$

Let s_9, s_{10}, s_{11} denote MWF 9–10, 10–11, 11–12; r_{10} denote TT 10–11, and w_{11} denote MW 11–12. The domains are

$$D(L_1) = \{s_9\}, \quad D(L_2) = \{s_9, s_{10}, s_{11}\},$$
$$D(T_1) = \{r_{10}\}, \quad D(T_2) = \{w_{11}\}.$$

We assume lecture room capacity two, tutorial room capacity one, and both tutorials shared by both lecture sections. The only nonzero soft penalty is 5 if both lecture sections occupy the same slot:

$$\text{Eval}(a) = 5 \cdot \mathbf{1}[a(L_1) = a(L_2)].$$

Suppose OR-tree search returns

$$q_0 = \{\, \text{Assigned}(L_1, s_9), \text{Assigned}(L_2, s_9),$$
$$\text{Assigned}(T_1, r_{10}), \text{Assigned}(T_2, w_{11})\}, \qquad \text{Eval}(a_{q_0}) = 5.$$

Both lectures can share the slot under the assumed room capacity, and neither tutorial overlaps them.

The controller considers the Move extension

$$\underbrace{\{\text{Assigned}(L_2, s_9)\}}_{A} \longrightarrow \underbrace{\{\text{Assigned}(L_2, s_{10})\}}_{B}.$$

This extension is valid because the new slot belongs to $D(L_2)$ and does not overlap either tutorial. Moving L_2 to s_{11} is not permitted, since that slot overlaps T_2 on Monday and Wednesday. Applying the extension yields

$$q_1 = (q_0 \setminus A) \cup B, \qquad \text{Constr}(a_{q_1}) = \text{true}, \qquad \text{Eval}(a_{q_1}) = 0 < 5.$$

Therefore our hill-climbing control accepts the extension and stops at zero penalty:

$$q_0 \xrightarrow{\text{Move } L_2: s_9 \to s_{10}} q_1, \qquad G_{set}(q_1) = \textsf{yes}.$$

This example shows how the set-based search takes a valid timetable from the OR-tree, applies a small change, and produces a better solution without violating any hard constraints.

5.6 Justification and limitations

We chose set-based search because it fits naturally with the second stage of our scheduling problem. Rather than rebuilding the timetable, we can focus on improving the assignments through smaller changes.

Our Move and Swap extensions also give us a straightforward way to explore different schedules, while the weight function allows us to compare them using the evaluation function defined in the problem.

However, the main limitation comes from our greedy search control.

Hill climbing only accepts extensions that improve the current timetable. This means we may eventually reach a local optimum, even when a better timetable exists elsewhere.

There is also the possibility that certain improvements require multiple components to move together. Since our current extensions only move one component or swap two, the search may not always be able to reach those solutions.

We could address this later through more advanced controls, such as simulated annealing, or by introducing extension rules that modify several assignments at once.

For our proposed model, however, greedy hill climbing gives us a relatively simple optimization process that follows the set-based search framework from Lecture 06 while keeping every intermediate timetable valid.

6 Proposal B: AND-Tree Optimization Following OR-Tree Feasibility

6.1 Search model: $A_\wedge = (S_\wedge, T_\wedge)$

The second stage receives a complete feasible timetable from OR-tree search and seeks a timetable with minimum soft-constraint penalty. We use the optimization interpretation of AND-tree search from Lecture 09: divide an optimization problem into cases, account for every case, and combine their results by comparison. Branch-and-bound avoids exploring a case when a sound lower bound proves that it cannot improve the best timetable already found.

Let a_{init} be the feasible timetable. The optimization stage initializes its incumbent and upper bound as

$$a_{\text{best}} = a_{\text{init}}, \qquad B = \text{Eval}(a_{\text{best}}). \tag{39}$$

The incumbent is always complete and satisfies every hard constraint. Its assignments are not imposed on the optimization tree. Let a_{fix} contain the mandatory assignments, including special bookings when triggered. Define the problem descriptions and their feasible completions by

$$\mathit{Prob}_\wedge = \{\text{Opt}(a, U) : a \supseteq a_{\text{fix}},\ U = X \setminus \text{dom}(a)\}, \tag{40}$$

$$\mathcal{F}(a) = \{\widehat{a} : X \to \mathit{Slots} : \widehat{a} \supseteq a,\ \widehat{a}(x) \in D(x),\ \text{Constr}(\widehat{a}) = \mathsf{true}\}, \tag{41}$$

$$V(a, U) = \min_{\widehat{a} \in \mathcal{F}(a)} \text{Eval}(\widehat{a}), \qquad \min \varnothing = +\infty. \tag{42}$$

Division relation. For an unresolved problem $p = \text{Opt}(a, U)$, choose $x \in U$ and let $D_a(x) = \{s_1, \ldots, s_k\}$ be its currently legal slots. Each child fixes one slot:

$$\mathit{Div}(p, p_1, \ldots, p_k), \qquad p_i = \text{Opt}(a \cup \{x \mapsto s_i\}, U \setminus \{x\}). \tag{43}$$

These cases partition the feasible completions of the parent; all cases must be accounted for, unlike OR-tree feasibility, which stops after one success. Thus

$$V(a, U) = \min_{1 \le i \le k} V(a \cup \{x \mapsto s_i\}, U \setminus \{x\}). \tag{44}$$

No independence between departments or courses is assumed: each child retains the full partial assignment.

Tree states and transitions. Following Lecture 09, $S_\wedge \subseteq \mathit{Atree}$ and $T_\wedge \subseteq S_\wedge \times S_\wedge$. A leaf is (p, sol); an internal node is $(p, \mathit{sol}, b_1, \ldots, b_k)$, with $\mathit{sol} \in \{?, \mathsf{yes}\}$. We instantiate the extension relation by

$$(p,?) \longrightarrow (p,\mathsf{yes}) \quad \text{when certified,} \qquad (p,?) \longrightarrow (p,?,(p_1,?),\ldots,(p_k,?)) \quad \text{when } \mathit{Div}(p,p_1,\ldots,p_k).$$

Extensions also act on leaves within larger trees; resolved subtrees may be contracted after retaining their result. These expansions and reverse contractions define $T_\wedge$, and the control never discards an unresolved case.

A resolved node v with problem $\mathrm{Opt}(a, U)$ retains a certificate ℓ_v such that

$$B \le \ell_v \le V(a, U). \tag{45}$$

For a complete valid case, the certificate is its exact evaluation (updating B first if necessary); for an infeasible case it is $+\infty$; for a pruned case it is $\mathrm{LB}(a)$; and when all children are resolved the parent retains $\min_i \ell_{v_i}$. A pruned case is *dominated*, not necessarily infeasible. Since B only decreases, retained certificates remain valid.

6.2 Search process: $P_\wedge = (A_\wedge, \mathit{Env}_\wedge, K_\wedge)$

The environment contains the scheduling instance, the checker Consistent, Eval, the incumbent a_{best}, the bound B, node certificates and the depth-first traversal records.

Lower bound and pruning. Let $w(x, s) \ge 0$ denote a stated preference penalty, $\mathcal{P}$ the requested unordered same-type offering pairs and $\mathcal{R}$ pairs of distinct lecture sections of the same course. For $A = \mathrm{dom}(a)$, the penalties already fixed by a partial assignment give

$$\begin{aligned}\mathrm{LB}(a) = &\sum_{x\in A}\sum_{s\in \mathit{Slots}} w(x,s)\,\mathbf{1}[a(x)\neq s] \\ &+ \mathit{pen}_{\mathit{notpaired}} \sum_{\substack{\{x,y\}\in\mathcal{P}\\ x,y\in A}} \mathbf{1}[a(x)\neq a(y)] \\ &+ \mathit{pen}_{\mathit{section}} \sum_{\substack{\{x,y\}\in\mathcal{R}\\ x,y\in A}} \mathbf{1}[a(x)= a(y)].\end{aligned} \tag{46}$$

Here $\mathbf{1}[Q]$ is 1 if Q holds and 0 otherwise. Each counted penalty persists in every completion. Minimum-occupancy penalties are omitted because additional offerings may remove a shortfall; they remain part of the full Eval. Thus for every $\widehat{a} \in \mathcal{F}(a)$,

$$\mathrm{LB}(a) \le \mathrm{Eval}(\widehat{a}), \qquad \boxed{\mathrm{LB}(a) \ge B \implies \text{prune the case.}} \tag{47}$$

Equality may be pruned because we need only one optimum.

Search control. The control $K_\wedge$ selects the deepest unresolved leaf (depth-first search). It first resolves complete or inconsistent cases, then rejects a case with an empty legal domain or $\mathrm{LB}(a) \ge B$. Otherwise it selects the next component by minimum remaining values,

$$x^* = \arg\min_{x\in U} |D_a(x)|, \qquad D_a(x) = \{s \in D(x) : \mathrm{Consistent}(a \cup \{x \mapsto s\})\}, \tag{48}$$

and divides into all legal slot cases. Children are visited by increasing lower bound, with fixed tie-breaks. After all children are resolved, their certificates are combined using (44). Every division assigns one unassigned offering, so depth is at most $|X|$.

Search instance, goal and implementation. The instance begins at

$$q_0 = (\mathrm{Opt}(a_{\mathrm{fix}}, X \setminus \mathrm{dom}(a_{\mathrm{fix}})), ?), \qquad G_\wedge(q) = \mathsf{yes} \iff \mathrm{Resolved}(q), \tag{49}$$

where a leaf is resolved when it holds a certificate and an internal node is resolved when all children are resolved. When the root is resolved, a_{best} is optimal; if interrupted by a resource limit, it remains feasible but is not proven optimal. If $B = 0$, nonnegativity of Eval already proves optimality.

For implementation, a **dictionary** maps offerings to slots, **sets/lists** hold remaining offerings and legal slots, and a **depth-first stack** stores one active case per level: partial assignments (or a reversible assignment trail), remaining alternatives, lower bound and certificate. A second dictionary or array caches per-slot occupancies, while a_{best} and B store the incumbent. Completed subtrees need not remain in memory after their certificates are retained.

6.3 Worked AND-tree derivation

Let $X = \{x, y, z\}$ be lectures of different courses with $D(x) = D(y) = D(z) = \{s_1, s_2\}$, where s_1 and s_2 are MWF 9–10 and 10–11. Both slots have capacity two and minimum zero. The only incompatibility is between x and y. All other penalties are zero except preferences $w(x, s_1) = 4$, $w(y, s_1) = 2$, $w(z, s_1) = 1$. Thus

$$\mathrm{Eval}(a) = 4\mathbf{1}[a(x) = s_2] + 2\mathbf{1}[a(y) = s_2] + \mathbf{1}[a(z) = s_2].$$

Suppose OR-tree search gives $a_{\mathrm{init}} = \{x \mapsto s_2, y \mapsto s_1, z \mapsto s_2\}$ with $B = 5$. Starting from the empty assignment, MRV chooses x (fixed tie-break). Division creates the $x \mapsto s_1$ and $x \mapsto s_2$ cases with lower bounds 0 and 4.

Visit	x	y	z	LB	Search-control action
1	s_1	–	–	0	Select y; only s_2 is legal.
2	s_1	s_2	–	2	Select z; both slots are legal.
3	s_1	s_2	s_1	2	Valid completion; update $B \leftarrow 2$.
4	s_1	s_2	s_2	3	Valid completion; incumbent stays 2.
5	s_2	–	–	4	Prune because $4 \geq B = 2$.

The resolved root retains $\ell_{\mathrm{root}} = \min(2, 4) = 2$. In general, complete partitions and sound certificates give

$$B \leq \ell_{\mathrm{root}} \leq V(a_{\mathrm{fix}}, X \setminus \mathrm{dom}(a_{\mathrm{fix}})) \leq B,$$

so resolution proves global optimality. The example returns $\{x \mapsto s_1, y \mapsto s_2, z \mapsto s_1\}$ with score 2; the search prunes the remaining case without claiming it infeasible. Worst-case work remains exponential.

7 Integration and Comparison

The OR-tree is the shared feasibility stage. The team proposes two alternatives, not a sequential three-stage pipeline. The shared OR-tree returns a feasible a_0, or FAILURE if every alternative has been exhausted. The chosen optimization branch is then applied:

$$a_{\mathrm{out}} = \begin{cases} \mathrm{SetBasedImprove}(I, a_0), & \text{Proposal A,} \\ \mathrm{ANDOptimize}(I, a_0), & \text{Proposal B.} \end{cases}$$

If feasibility search is interrupted without a solution, the outcome is inconclusive rather than infeasible.

Table 2: Summary of proposed models

	OR-tree	Set-based	AND-tree
Purpose	Feasibility	Local improvement	Exact optimization
State	OR-tree	Assignment fact set	AND-tree
Primitive	*Altern*	*EXT*	*Div*
Control	DFS + MRV	Hill climbing	Branch-and-bound
Optimal?	Not targeted	Not guaranteed	If complete

Both alternatives preserve hard constraints and return a timetable with evaluation no greater than the OR-tree starting solution. Proposal A is typically less memory-intensive but can stop at a local optimum; Proposal B can prove optimality if its search completes, but has exponential worst-case work. If its resource budget expires, it returns the best feasible incumbent without claiming optimality. The two examples use different small instances; each demonstrates the relevant model and its control.

8 Discussion and Limitations

The shared OR-tree stage may exhaust all alternatives without finding a feasible timetable. If it is interrupted before that determination, infeasibility has not been established. Set-based hill climbing can reach local minima because it only accepts improving Moves or Swaps. AND-tree branch-and-bound preserves a complete feasible incumbent while exploring the full assignment space, but its worst-case work is exponential; only a completed search or a zero lower bound with zero penalty establishes global optimality. For larger instances, a time budget may be needed, and the returned timetable should be labelled feasible rather than optimal when the search is interrupted.

9 Conclusion

We proposed two complete timetabling pipelines sharing an OR-tree feasibility stage. The set-based branch offers simple, feasibility-preserving local improvement, whereas AND-tree branch-and-bound searches across alternative assignments and can prove an optimum when completed. Both use the same hard-constraint checker and evaluation objective. The proposed models and worked derivations specify how each search process can be instantiated for the course timetabling problem.

References

[1] CPSC 433. *Project Description*, Version 1, University of Calgary, Fall 2025.

[2] CPSC 433. *Project Problem Description*, Version 1, University of Calgary, Fall 2025.

[3] CPSC 433. *Lecture 05: Search*, University of Calgary.

[4] CPSC 433. *Lecture 06: Set-Based Search*, University of Calgary.

[5] CPSC 433. *Lecture 09: And-Tree-Based Search*, University of Calgary.

[6] CPSC 433. *Lecture 11: Or-Tree-Based Search*, University of Calgary.

[7] CPSC 433. *Lecture 14: Search Controls*, University of Calgary.